

Partie I : robotique

Cours 0 — introduction

L3 informatique, Sorbonne Université

UE IA et Jeux — LU3IN025

Partie Robotique et IA

Nicolas Bredeche

Sorbonne Université

ISIR, UMR 7222

Paris, France

nicolas.bredeche@sorbonne-universite.fr

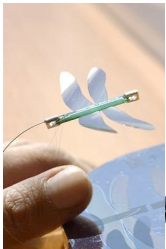


Metropolis Fritz Lang (1926)



présentiel, 2024-2025

2



(DGA/COM)



Atlas (Boston Dynamics / Google)



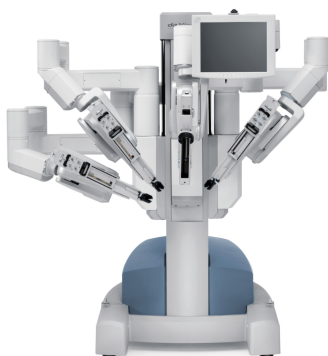
Spirit et Sojourner (NASA)



Stanley (Stanford) — Grand challenge DARPA 2005



S-bots(Swarmbots EU)



Robot Da Vinci



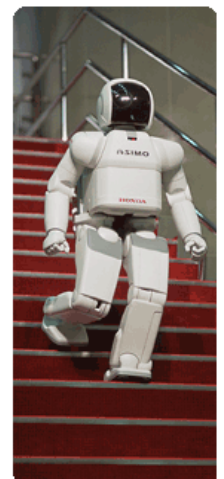
Schaft (KAIST / Google)



Aibo ERS7 (Sony)



Roomba (i-Robot)



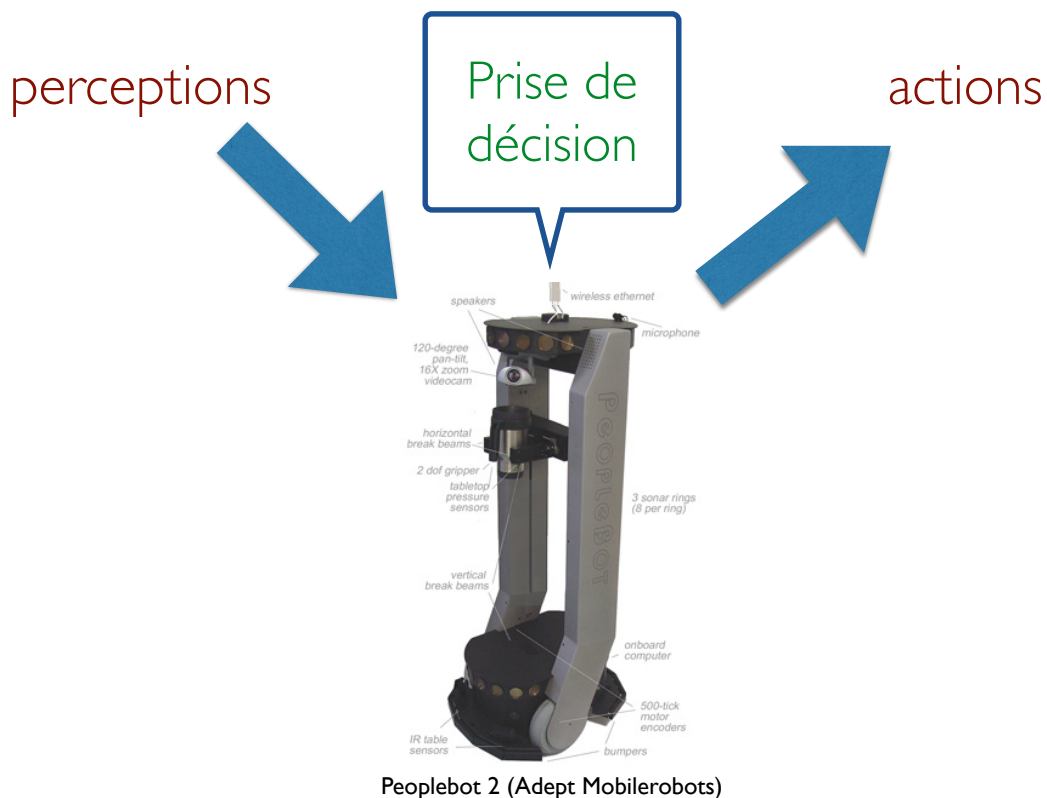
Asimo (Honda)



- Communication
 - Réseau sans fil (WIFI)
 - Microphone
 - Haut-parleurs
- Vision
 - Caméra couleur/n&b/linéaire/stéréo
 - Fonction Zoom
 - Boîtier de commande Pan-Tilt
- Senseurs de proximité
 - Ceinture de sonars ou IR
 - Bumpers (de contact)
- Moteur (servo-moteurs, muscles artificiels, ...)
 - Bras manipulateur
 - Moteur (avec odomètre)
- Autres
 - PC embarqué (Linux) au format PC104+
 - Autonomie énergétique (Batterie rechargeable)
 - Compas
 - Gyroscope
 - Accéléromètre
- etc.

nicolas.bredeche@upmc.fr

Peoplebot 2 (Adept MobileroBots)



nicolas.bredeche@upmc.fr

perceptions



Prise de
décision



actions



Nao (Aldebaran Robotics)

nicolas.bredeche@upmc.fr

perceptions



Prise de
décision



actions



Schaft (Google)

nicolas.bredeche@upmc.fr

perceptions

Prise de
décision

actions



Roomba (iRobot)

nicolas.bredecche@upmc.fr

perceptions

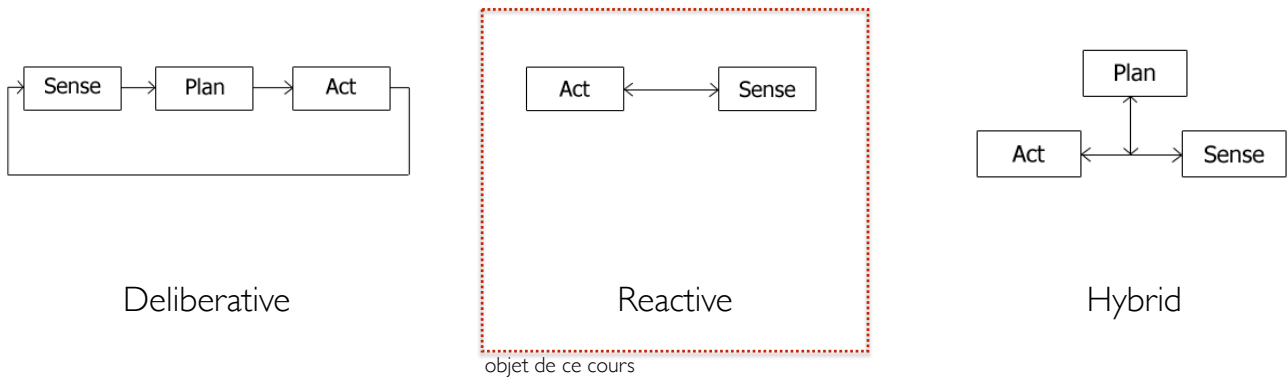
Prise de
décision

actions



"Blinky" (Pacman)

nicolas.bredecche@upmc.fr



Contenu et objectif des trois prochains cours

- Robotique et agent(s) autonome(s)
 - Un **robot** peut être vu comme un **agent**
 - problèmes propres à la robotique *contrôle optimal, cartographie, traitement de données en situation réelle, problème de sécurité, etc.*
 - problème plus généraux liés à la notion d'agent: *conception de comportements individuels, de comportements collectifs, apprentissage et optimisation*
 - Autres exemples d'agents: *simulation d'individus, personnages de jeux vidéos, etc.*
- Objectif de cette partie du cours:
 - Donner les outils pour la conception (manuelle et automatique) de comportement d'agent autonome
 - Au delà du contenu robotique: les agents autonomes, seuls ou en groupe

- **Cours**

- Cours 1 : “Architecture de comportements”
- Cours 2 : “Algorithme génétique et application à la robotique”
- Cours 3 : cours d'ouverture + Q/R projet

- **Pratique**

- TME 1 : programmation de comportements pour robot mobile
- TME 2 : algorithme évolutionniste et robotique
- Soutenance mini-projet — plus de détails à la fin de ce cours

Organisation des TME:

- 2h travail pratique
- 2h sur projet

Projet en binôme

nicolas.bredeche@upmc.fr

Cours 1 — Architectures de comportement

L3 informatique, Sorbonne Université

UE IA et Jeux — LU3IN025

Partie Robotique et IA

Nicolas Bredeche

Sorbonne Université

ISIR, UMR 7222

Paris, France

nicolas.bredeche@sorbonne-universite.fr

...



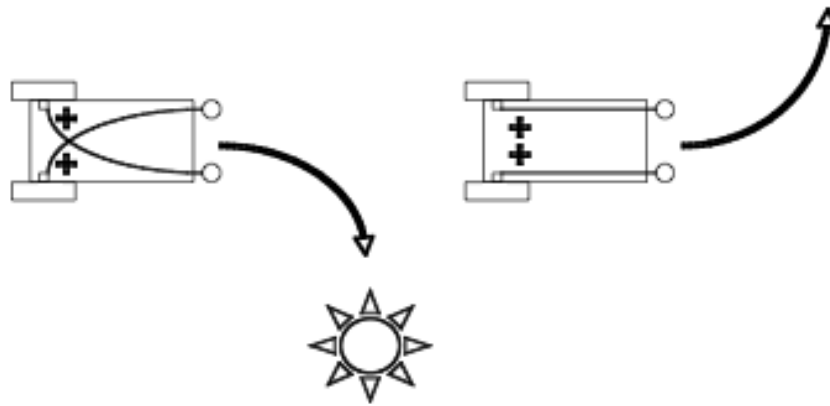
Metropolis Fritz Lang (1926)

- Architectures de contrôle
 - ▶ Véhicules de Braitenberg
 - ▶ Automates à états finis
 - ▶ Arbres de décision
 - ▶ Arbres de comportements
 - ▶ Architecture de Subsumption

nicolas.bredeche@upmc.fr

Comportements réactifs

comportements simples, dynamiques complexes



Comportements de "Braitenberg"

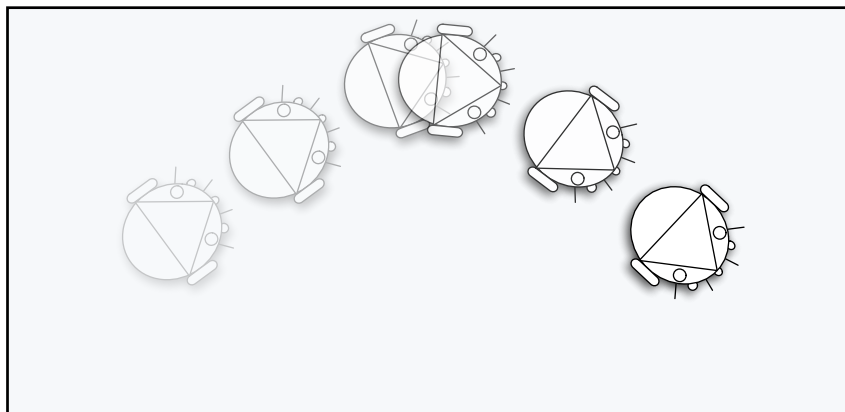
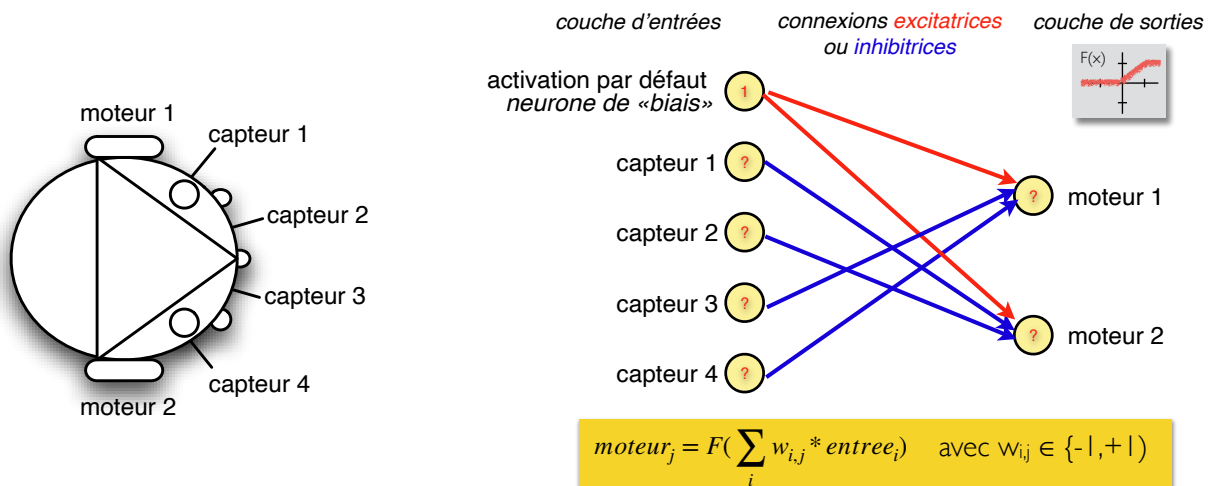
$$\text{moteur}_j = \sum_i p_i * \text{entree}_i \quad \text{avec } p_i \in \{-1, 0, +1\}$$

connexion
inhibitrice

connexion
excitatrice

pas de connexion

Exemple: robot-éviteur

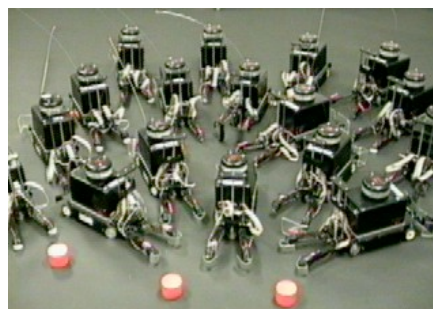
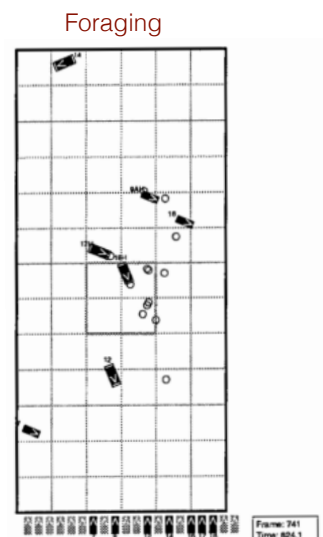
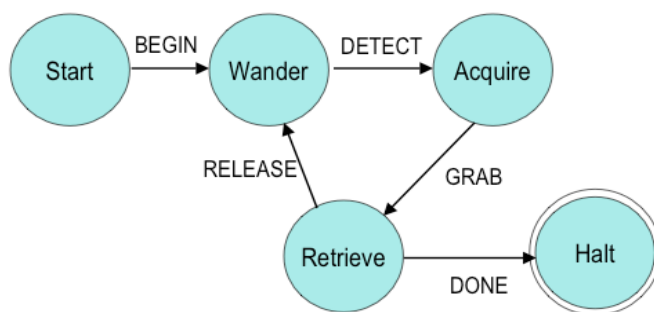


Automates à états finis

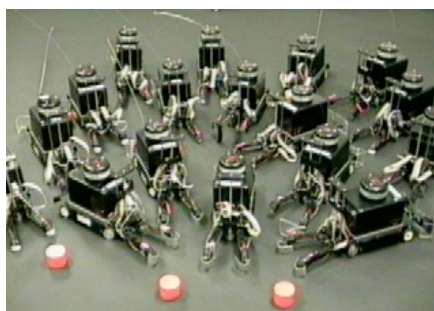
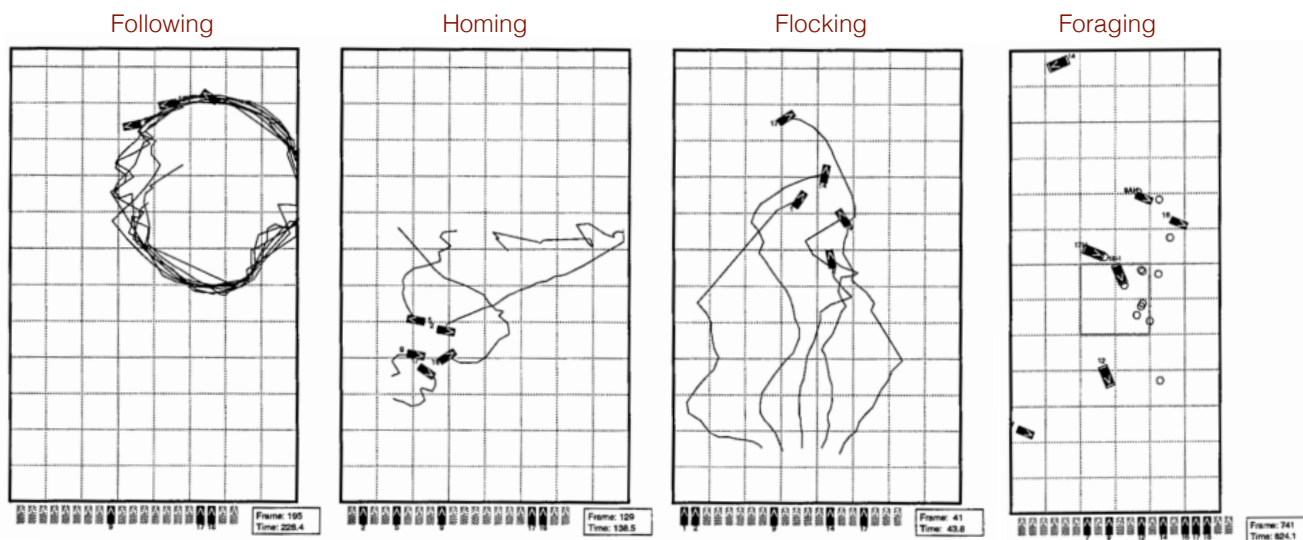
UE LU3IN025 “IA et Jeux”, partie Robotique — Cours I.B

Machine à états finis

18



The « nerd herd »



The « nerd herd »

nicolas.bredecche@upmc.fr

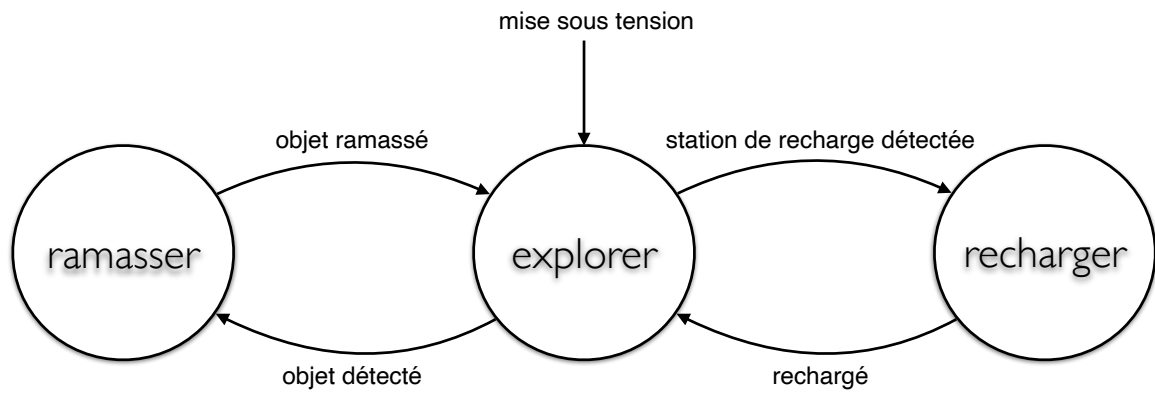
Mataric (1994) Ph.D. Thesis



Demo avec des robots

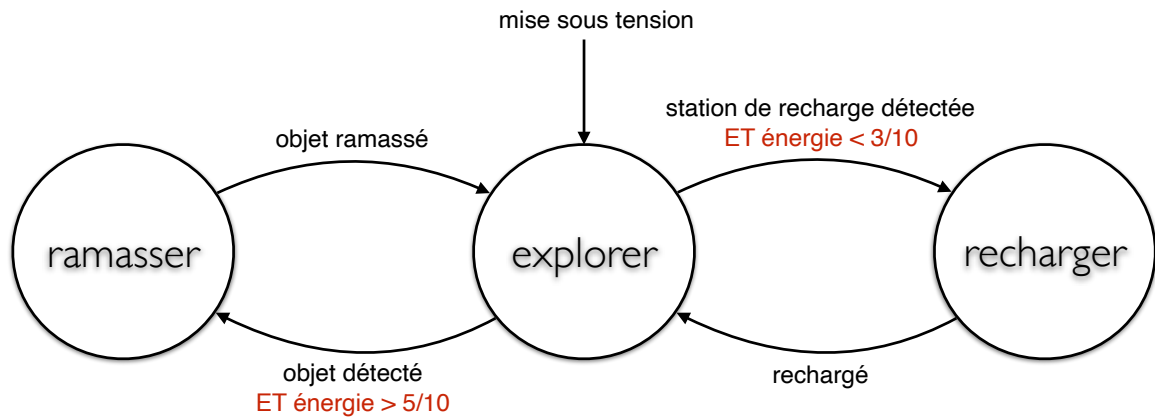
1. Des comportements réactifs (“jaune”, “rouge” et “vert”)
2. Un comportement collectif : suivi de leader

Automate à états finis



Permet d'implémenter une certaine forme de mémoire

Automate à états finis



Variable interne
- niveau d'énergie

Utilisation de variables internes

Q: Qu'est ce qui caractérise un comportement de type Braintenberg?

1. utilisation de structure if-then-else
2. utilisation d'opérateurs de type multiplication
3. utilisation d'opérateurs de type addition
4. utilisation de facteurs positif et négatif

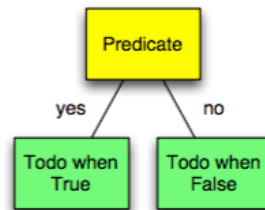
Q: plusieurs robots explorant l'environnement pour ramasser des objets, c'est du...

1. fouragement
2. vol en formation
3. suivi de leader
4. agrégation

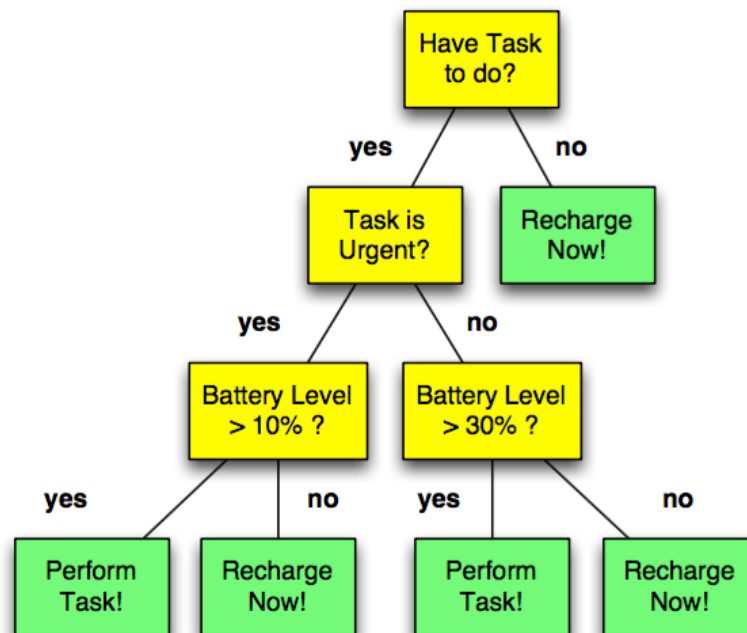
Q: soit 5 robots faisant du suivi de leader; combien de comportement(s) différent(s) faut-il programmer?

1. un
2. deux
3. cinq
4. autre réponse

Arbres de décision

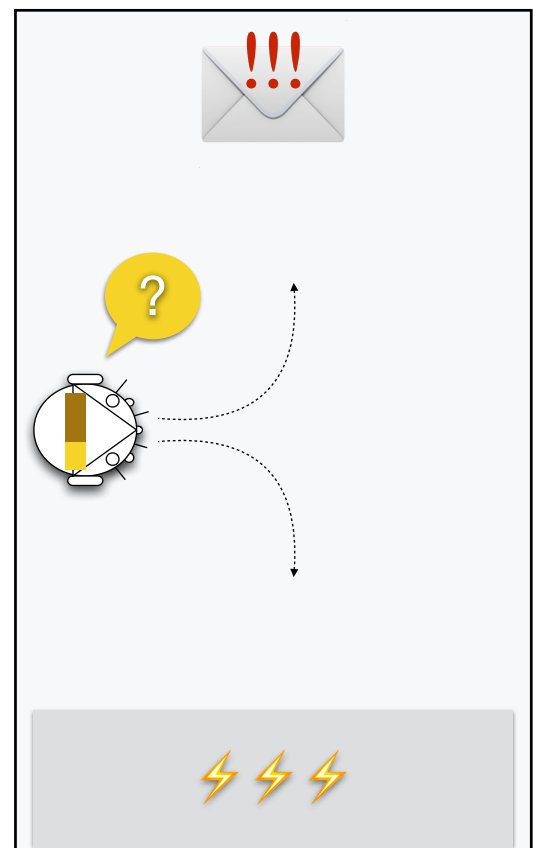
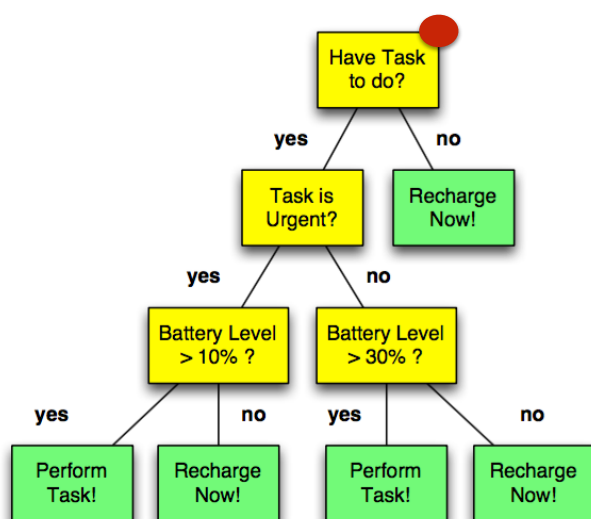
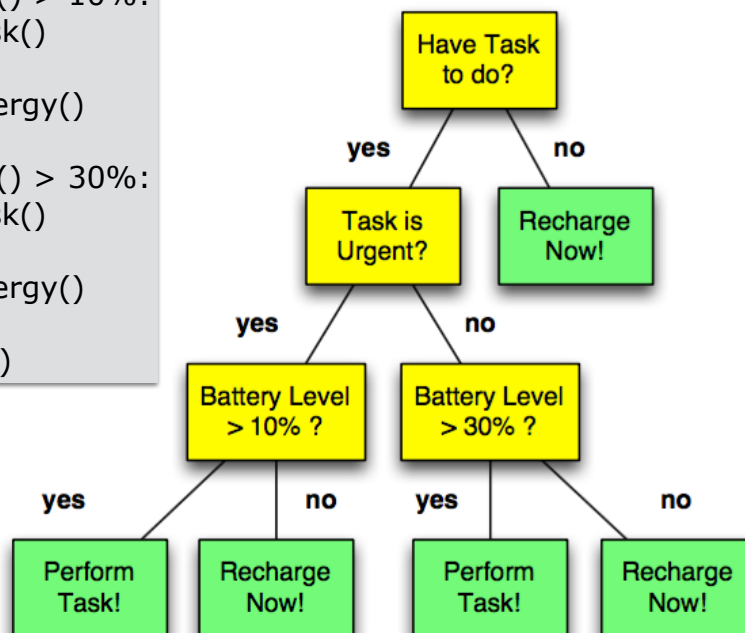


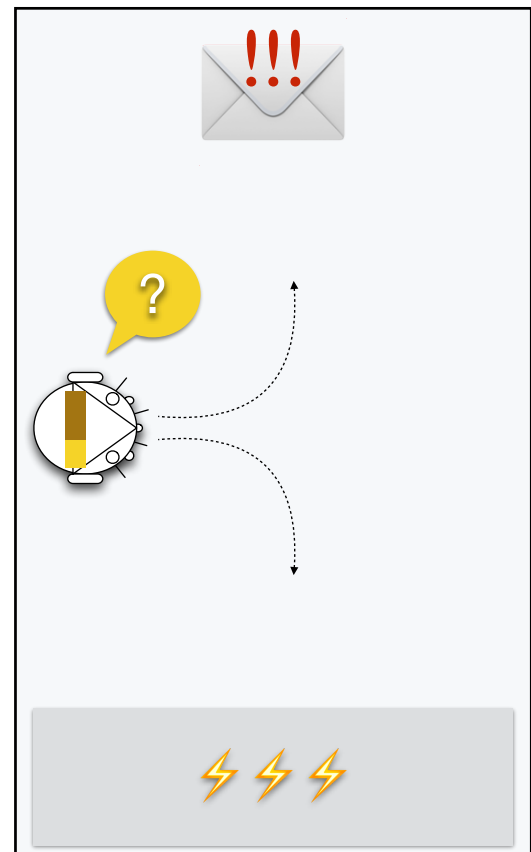
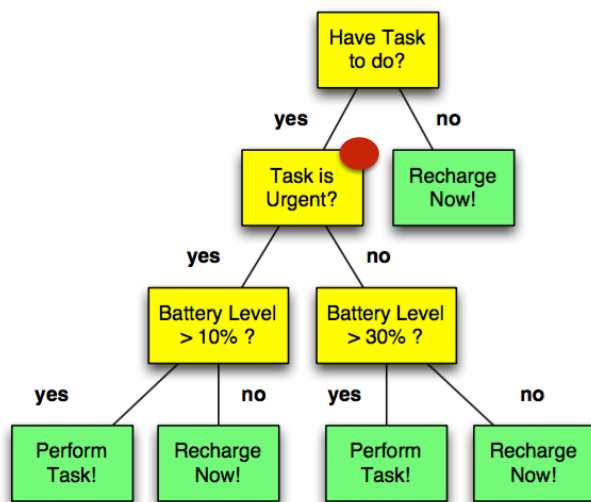
A chaque pas de temps, l'arbre est parcouru



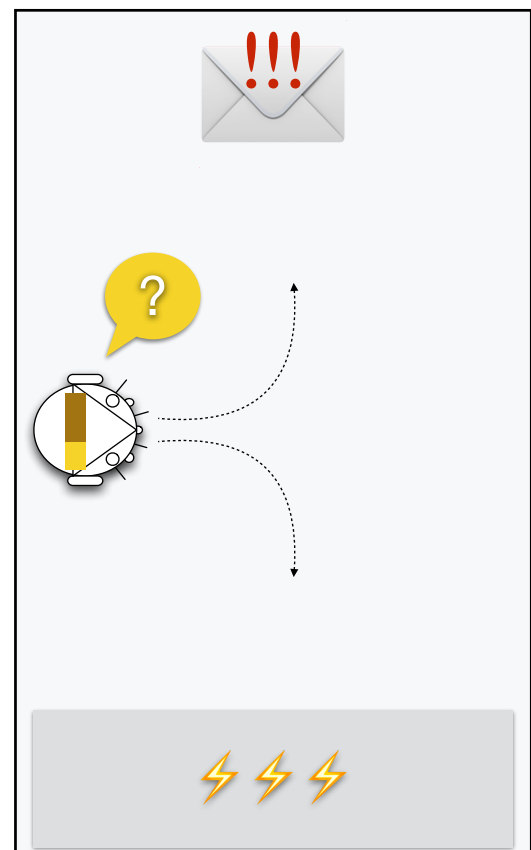
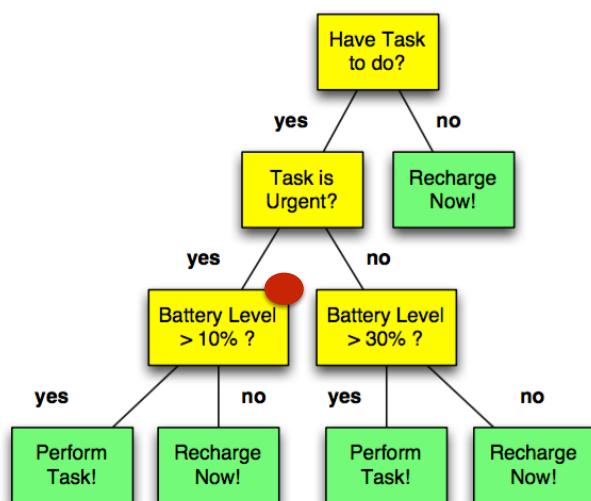
```

if task.isActive() == True:
  if task.isUrgent() == True:
    if getBatteryLevel() > 10%:
      moveTowardTask()
    else:
      moveTowardEnergy()
  else:
    if getBatteryLevel() > 30%:
      moveTowardTask()
    else:
      moveTowardEnergy()
else:
  moveTowardEnergy()
    
```

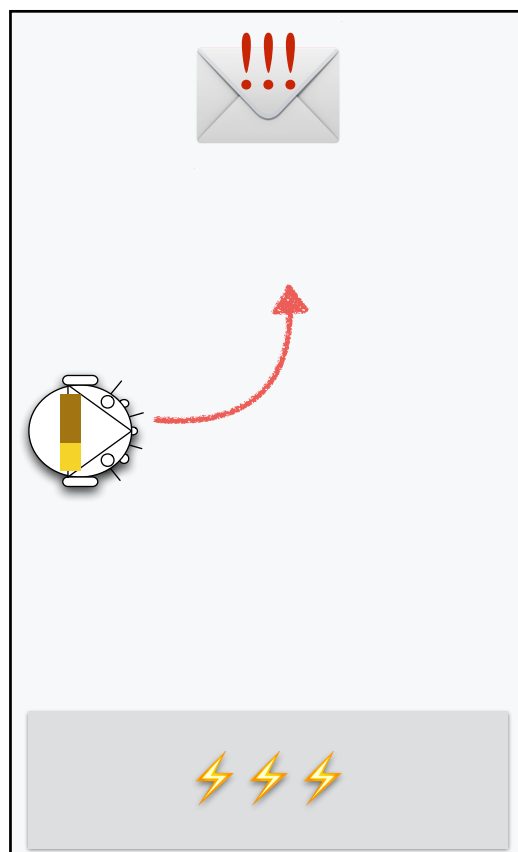
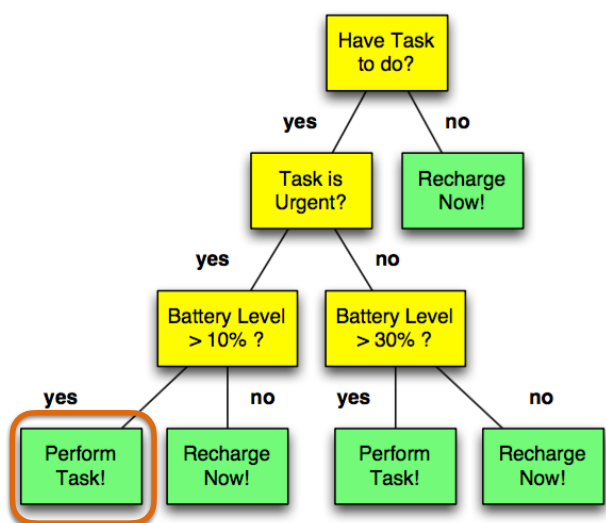




nicolas.bredecche@upmc.fr

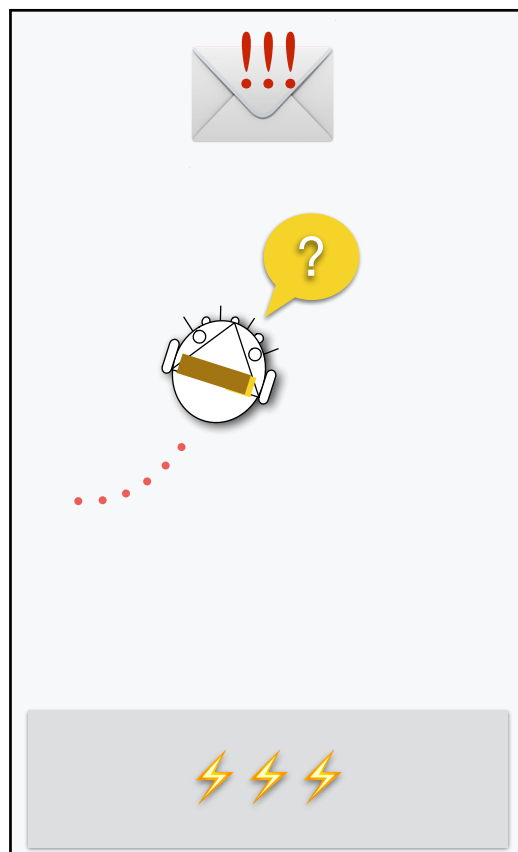
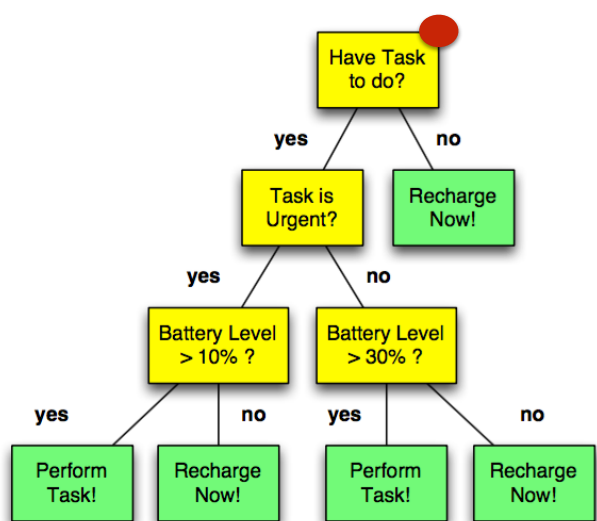


nicolas.bredecche@upmc.fr

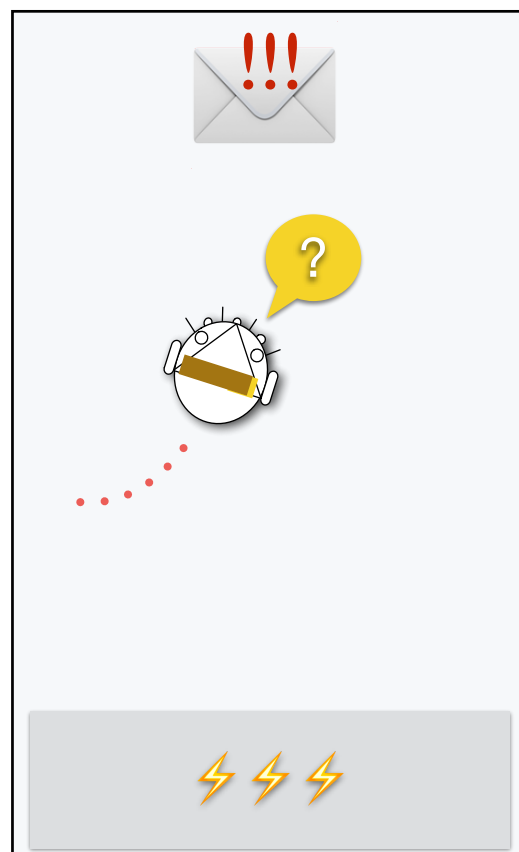
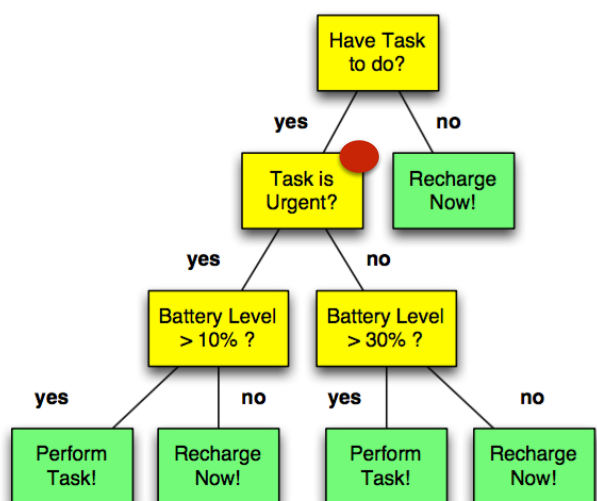


nicolas.bredecche@upmc.fr

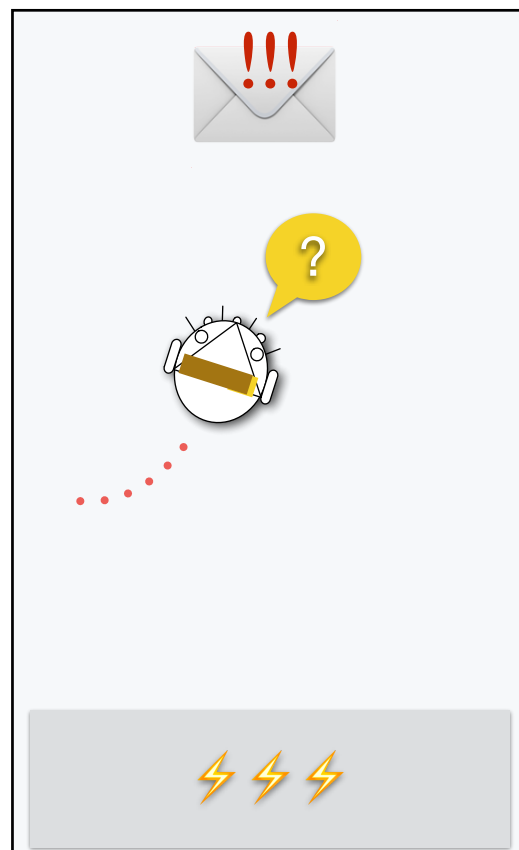
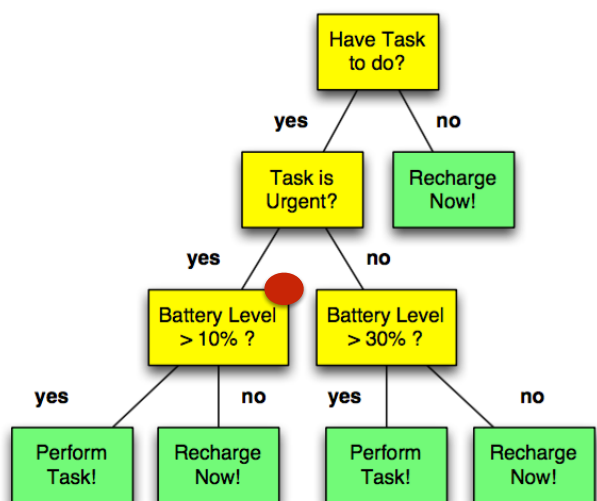
un peu plus tard...



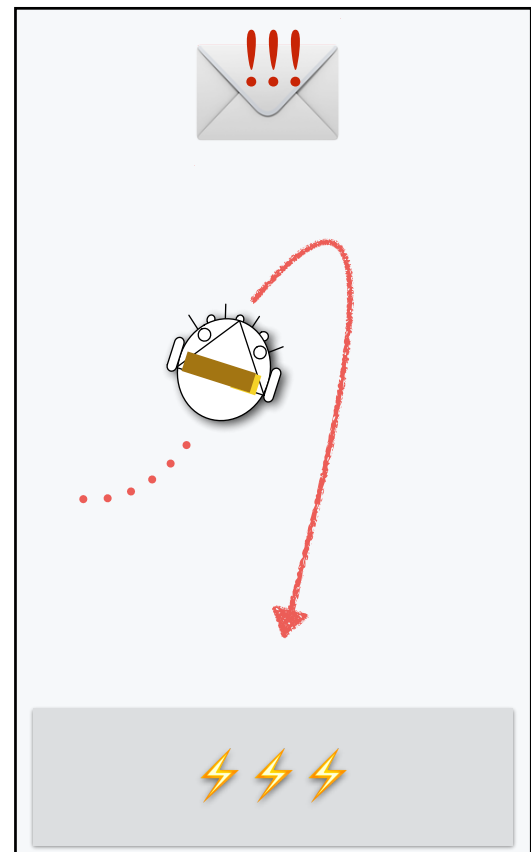
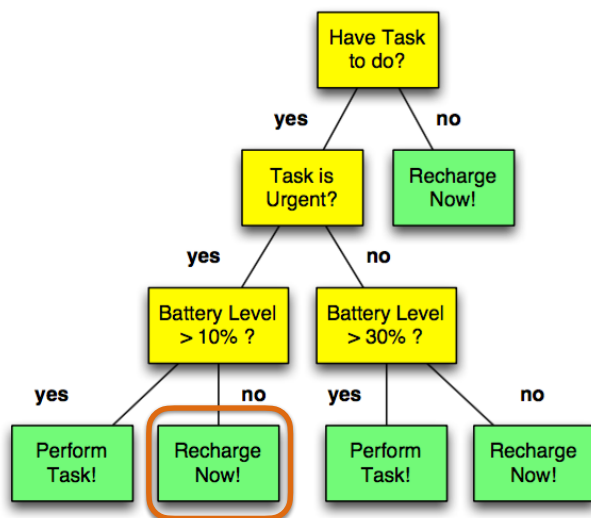
nicolas.bredecche@upmc.fr



nicolas.bredecche@upmc.fr



nicolas.bredecche@upmc.fr



nicolas.bredecche@upmc.fr

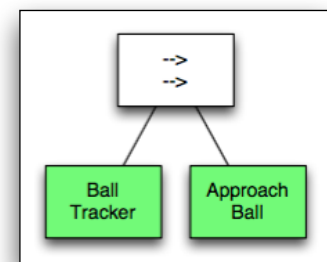
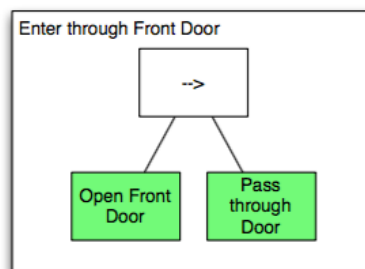
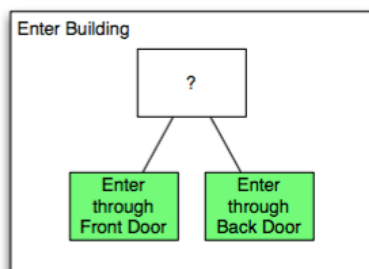
Arbres de comportements

- Graphe orienté acyclique (arbre avec une racine et des arcs orientés)
 - Noeuds internes: fallbacks, sequences, parallels
 - Feuilles: actions, conditions
 - retours possibles: “running”, “success”, “failure”
- Processus
 - A chaque pas de temps
 - soit (au moins) une feuille est exécutée
 - soit l'intégralité de l'arbre a été parcouru (donc: la racine retourne “failure”)
 - Au début d'un pas de temps
 - on (re-)commence à la racine de l'arbre

nicolas.bredeche@upmc.fr

Arbre de comportements

40

**Fallback****Fonctionnement:**

exécute A ; si **échec**, exécute B

Condition d'arrêt:

lorsqu'une action est réalisée avec succès

Condition d'échec:

si toutes les actions sont en échec

Exemples:

Ex.1 j'ouvre la porte, sinon, la fenêtre
Ex.2 il fait jour? sinon, je dors

Sequence**Fonctionnement:**

exécute A; si réussi, exécute B **ensuite**

Condition d'arrêt:

lorsque toutes les actions sont réalisées avec succès

Condition d'échec:

dès qu'une action est en échec

Exemples:

Ex.1: j'ouvre la porte puis la fenêtre
Ex.2: s'il y a une porte, j'ouvre la porte

Parallel**Fonctionnement:**

A et B en **même temps**

Condition d'arrêt:

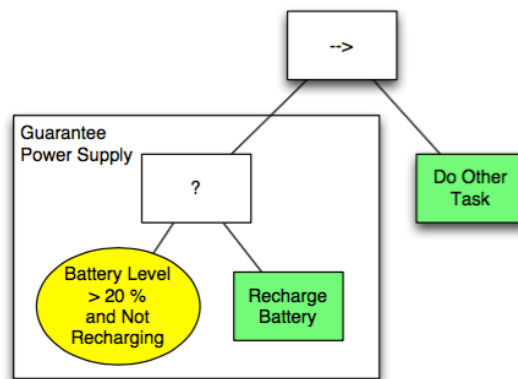
lorsque toutes les actions ont été réalisées (avec succès ou non)

Condition d'échec:

si L actions (parmi N) sont en échec (L à définir)

Exemples

Ex.1 j'ouvre la porte et la fenêtre
Ex.2 je cours et je saute

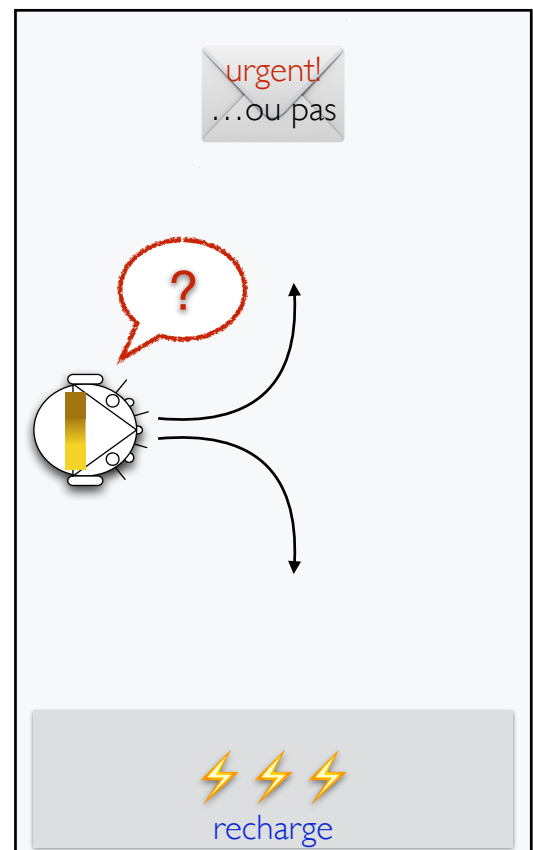
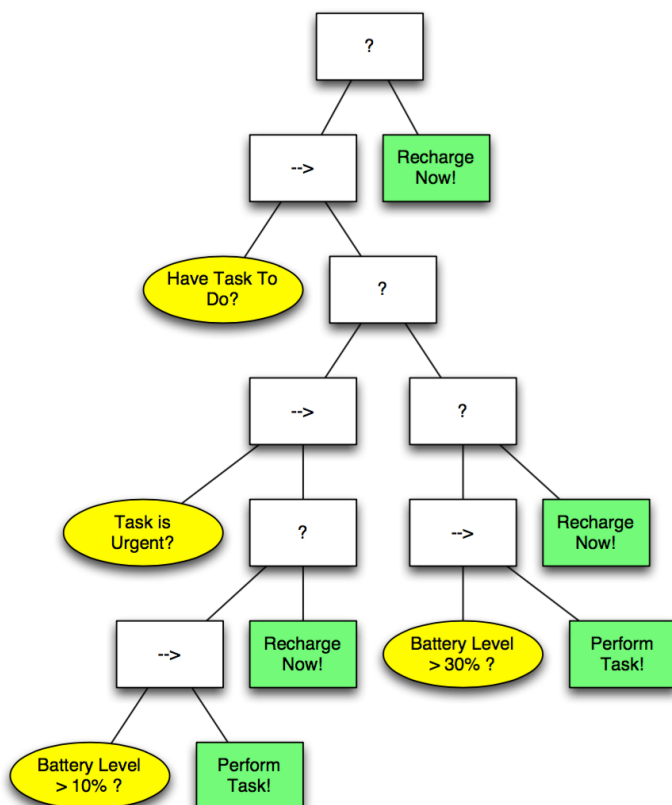


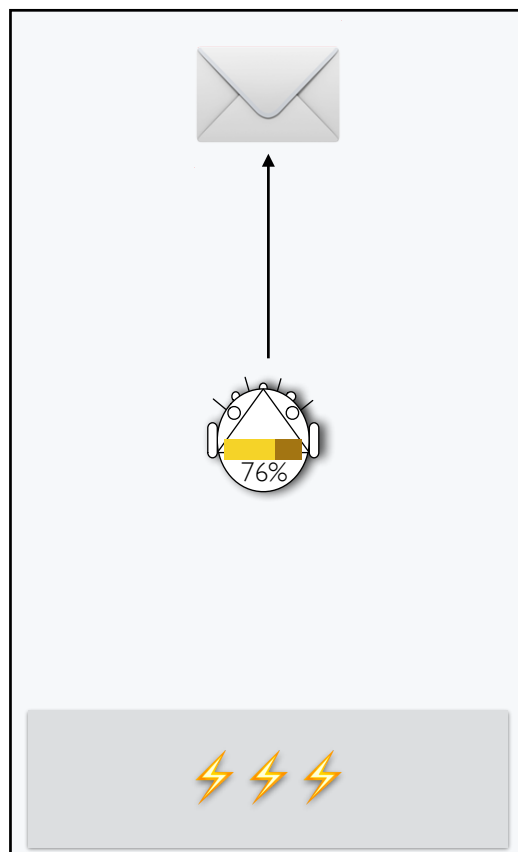
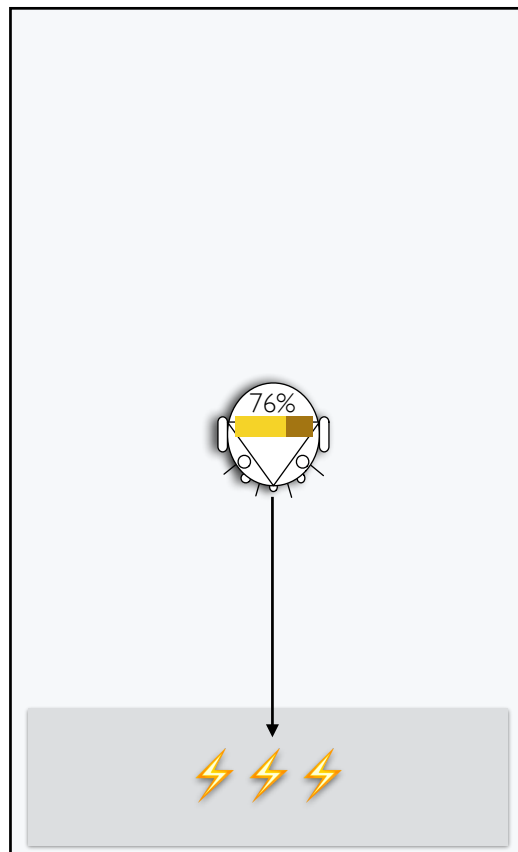
Condition

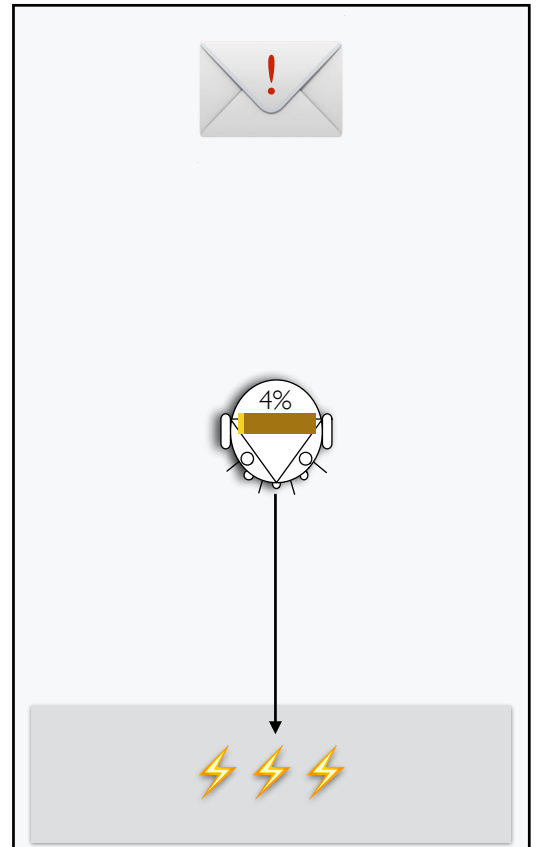
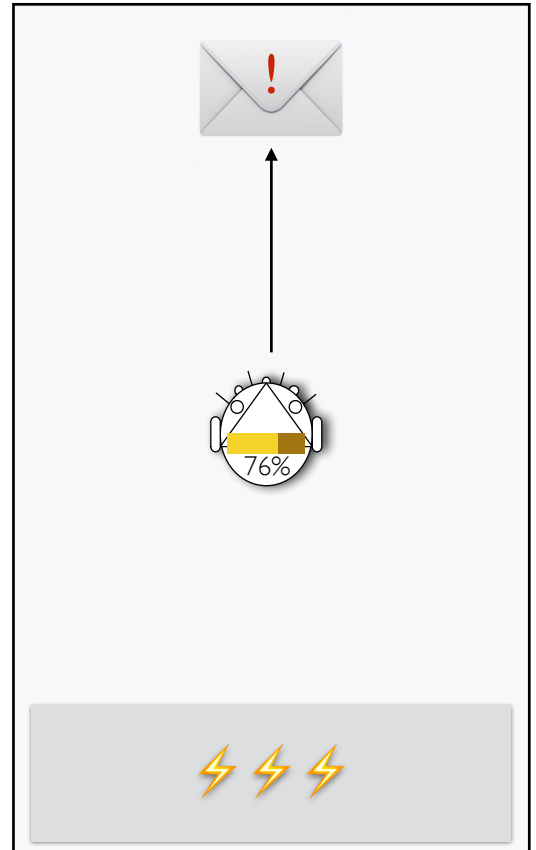
à chaque pas de temps:
si A est vrai, n'exécute pas B
puis: exécute C

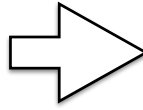
Remarques:
ici, B reste à l'état "running" tant
que la recharge n'est pas terminée

Optimisation de l'arbre de comportements

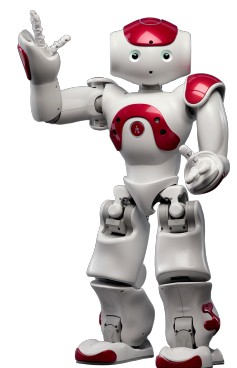


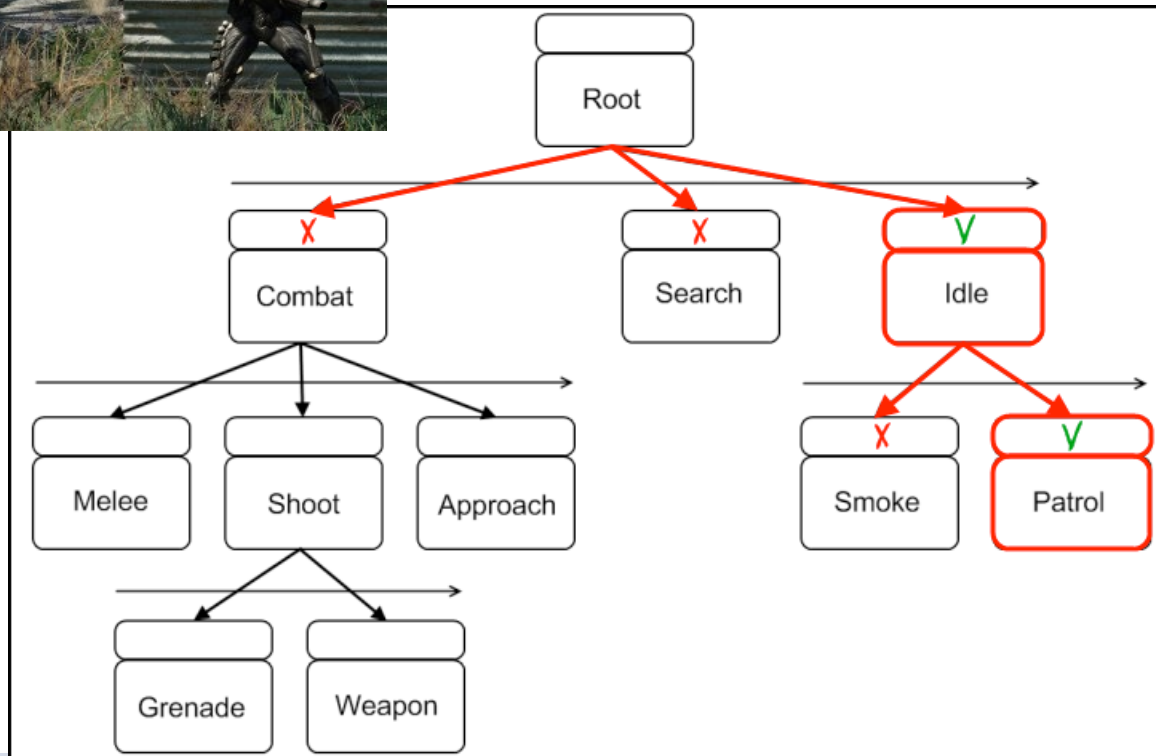






48





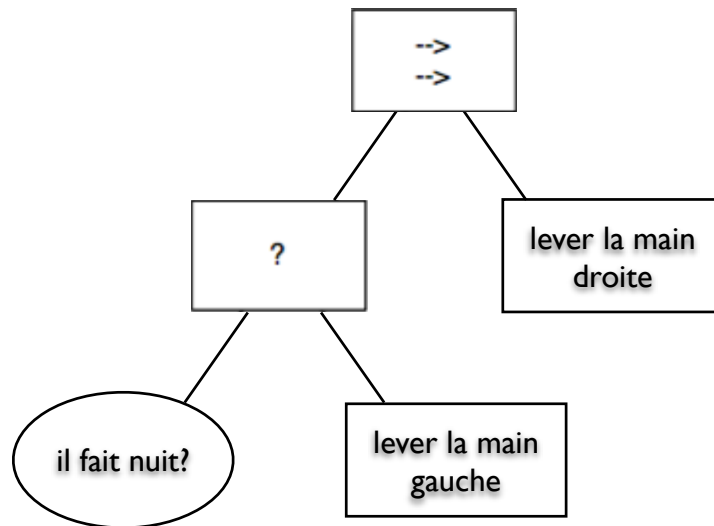
source: Pillosu, Algamedev Paris 2009

QUIZZ

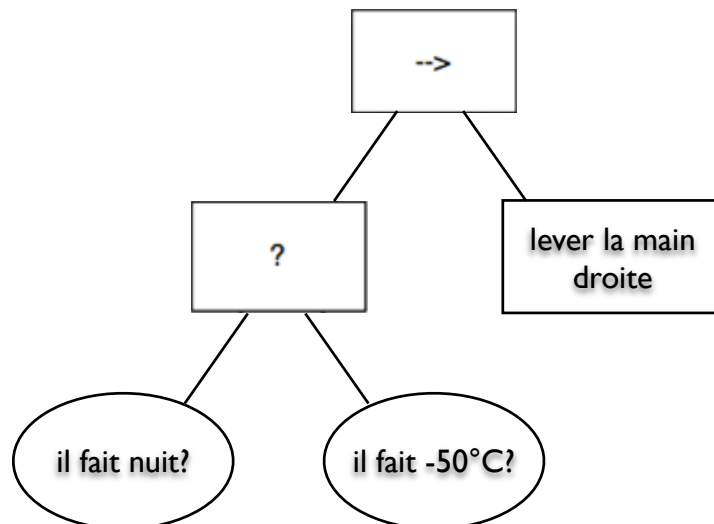
50

Q: quel est la représentation qui permet de modéliser des comportements les plus complexes?

1. arbre de décision
2. arbre de comportement
3. aucune des deux



Q: exécutez cet arbre de comportement



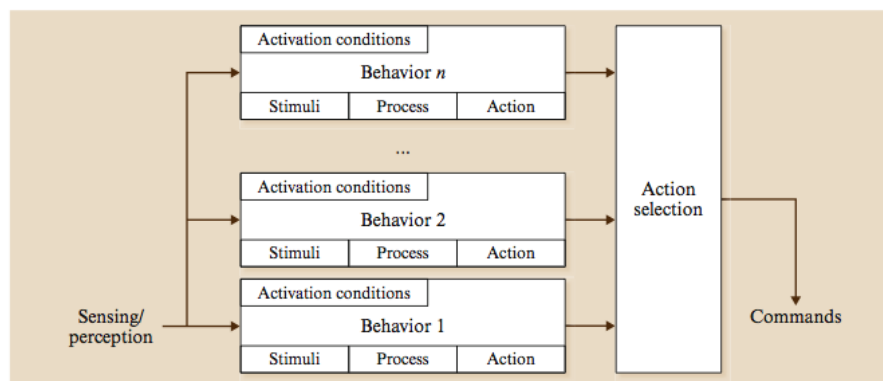
Q: exécutez cet arbre de comportement

Architecture de subsomption

UE LU3IN025 “IA et Jeux”, partie Robotique — Cours I.E

Architecture de subsomption

54



credits: handbook of robotics, chap 38

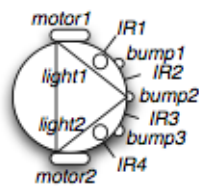
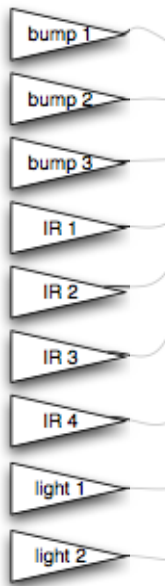
- **Eléments**

- ▶ Une bibliothèque de **comportements**, organisé par ordre de priorité
- ▶ Mécanisme de **sélection d'action** parmi les comportements activables

- **Fonctionnement**

- ▶ Le comportement actif de plus haut niveau prend la main

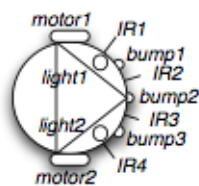
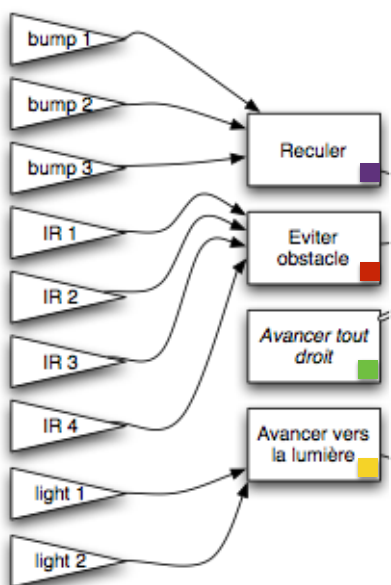
senseurs



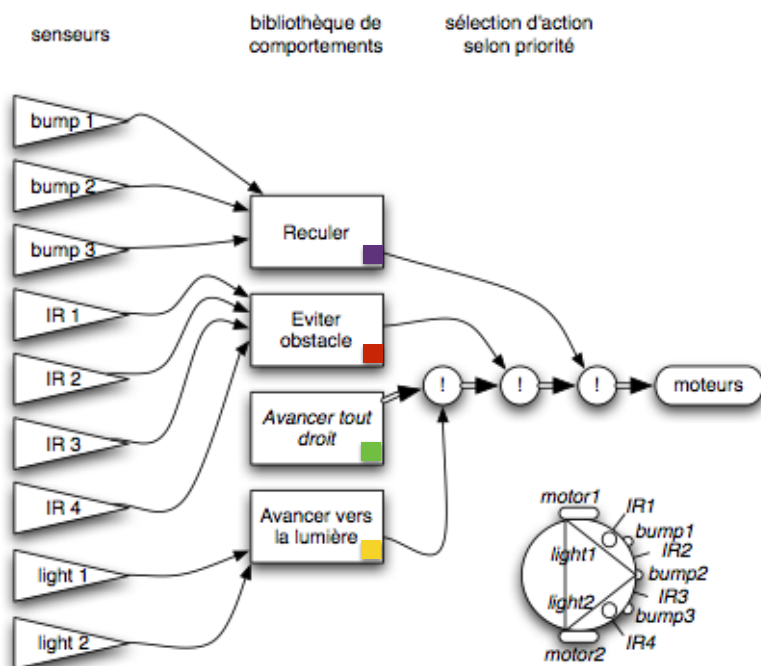
nicolas.bredeche@upmc.fr

senseurs

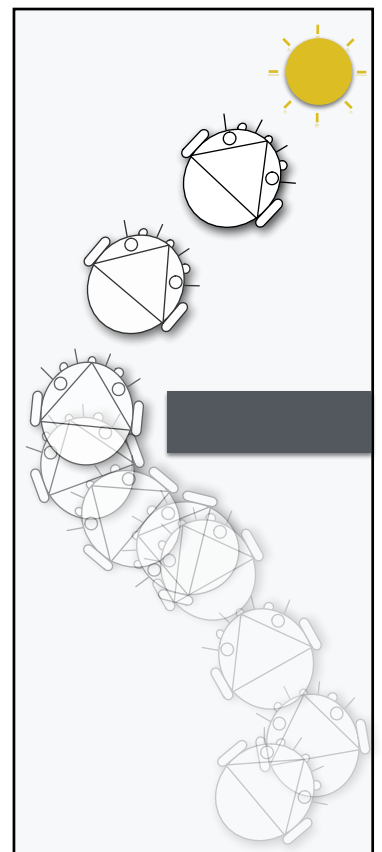
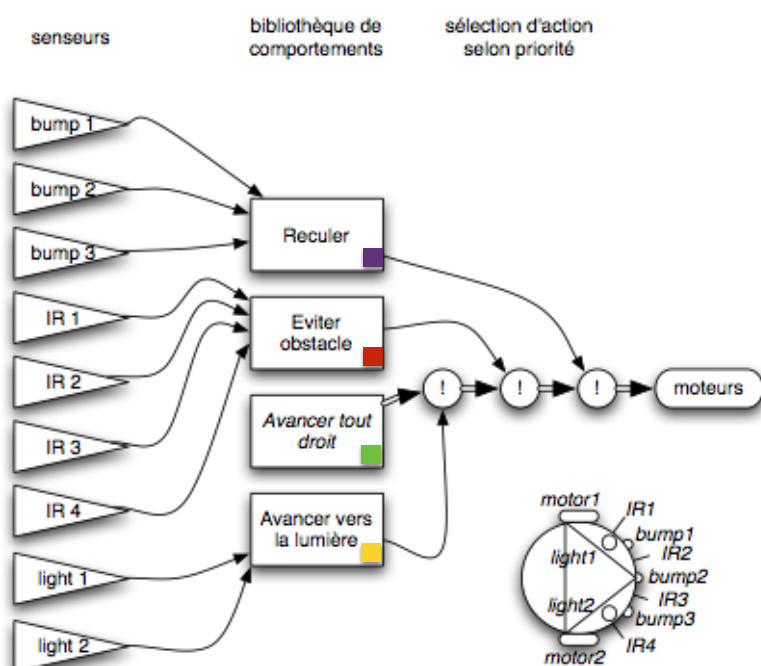
bibliothèque de
comportements



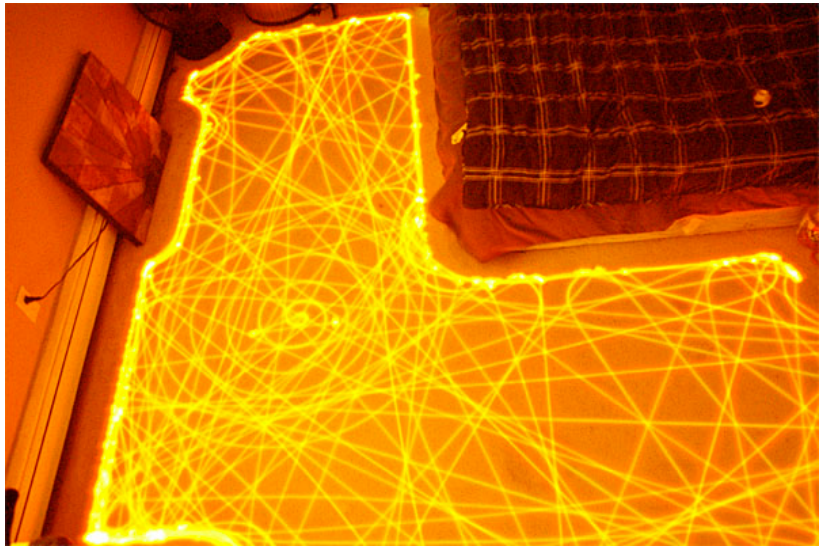
nicolas.bredeche@upmc.fr



nicolas.bredeche@upmc.fr



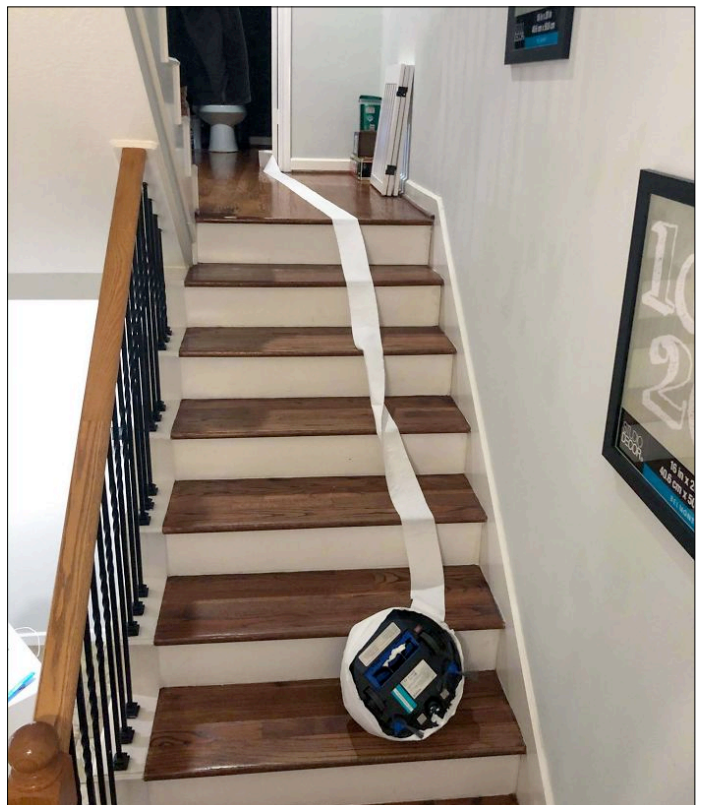
nicolas.bredeche@upmc.fr



nicolas.bredecche@upmc.fr

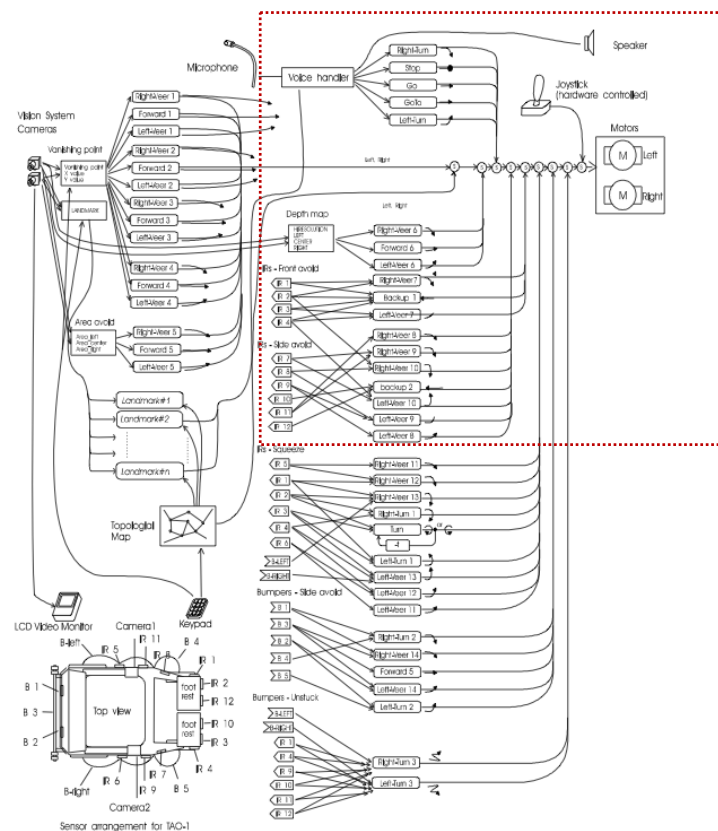
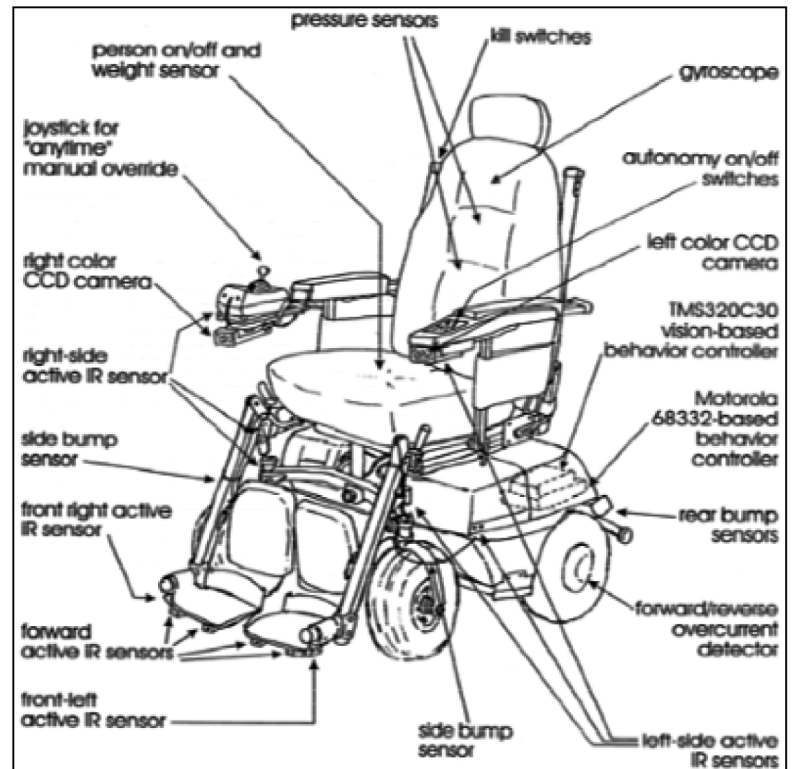
source: <http://signaltheorist.com/?p=91>

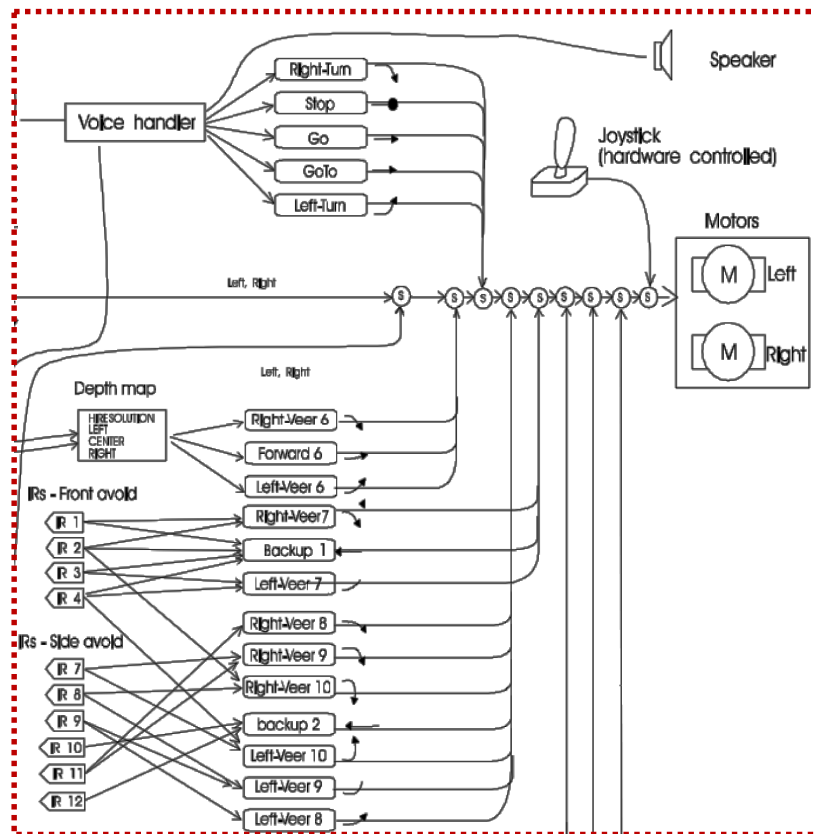
60



nicolas.bredecche@upmc.fr

source: <https://pleated-jeans.com/2020/11/09/roomba-fails/>



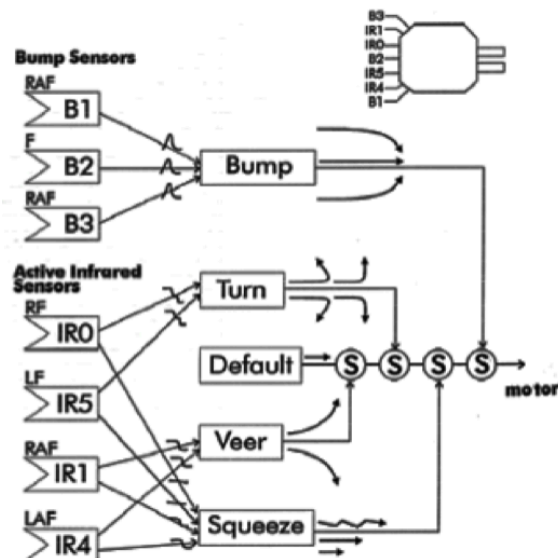


nicolas.bredeche@upmc.fr

TAO-I wheelchair (Applied AI System) [Gomi, 1996]

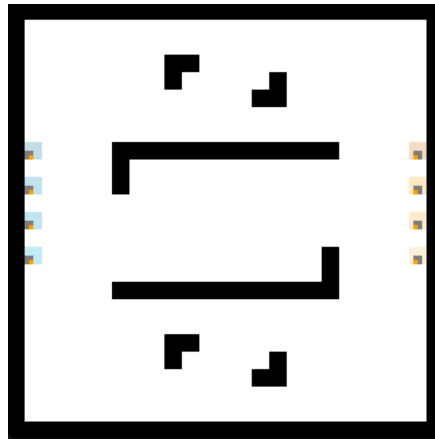
QUIZZ

64



Q: quel sont les capteurs qui activent le comportement le plus prioritaire?

1. bumpers (senseurs de contact)
2. Infrared sensor (senseurs de proximité)
3. moteurs

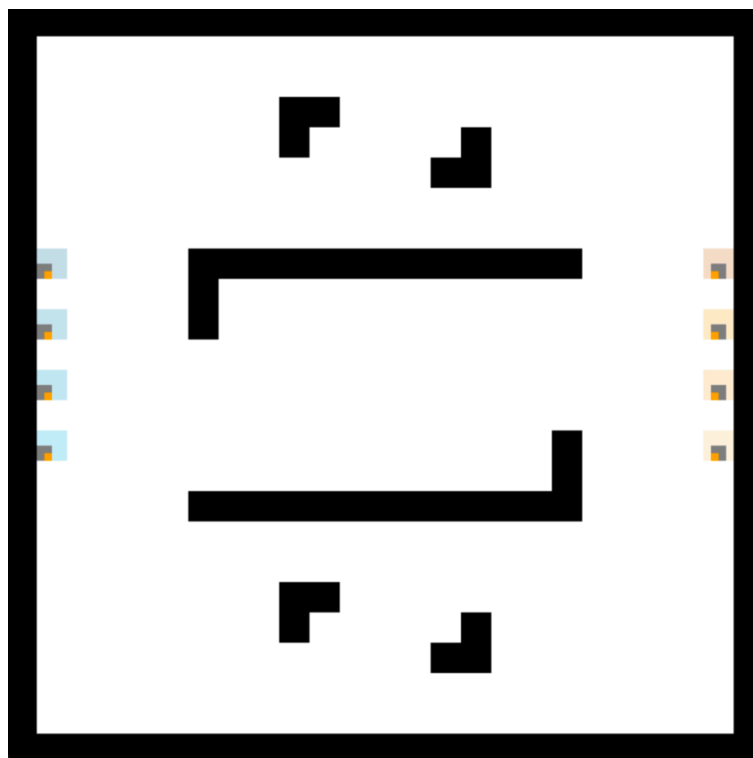


Projet

Paint Wars

66

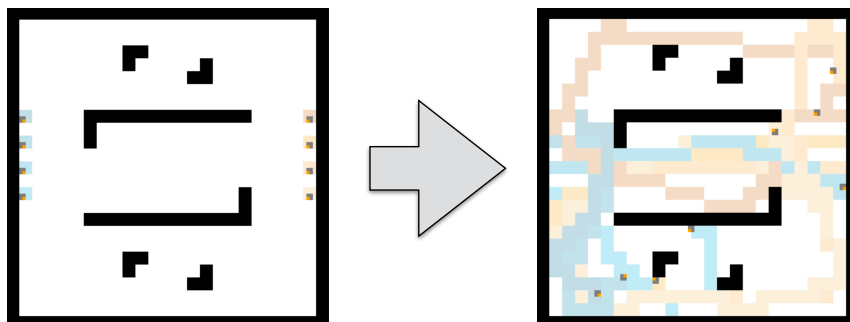
4 robots
équipe A



4 robots
équipe B

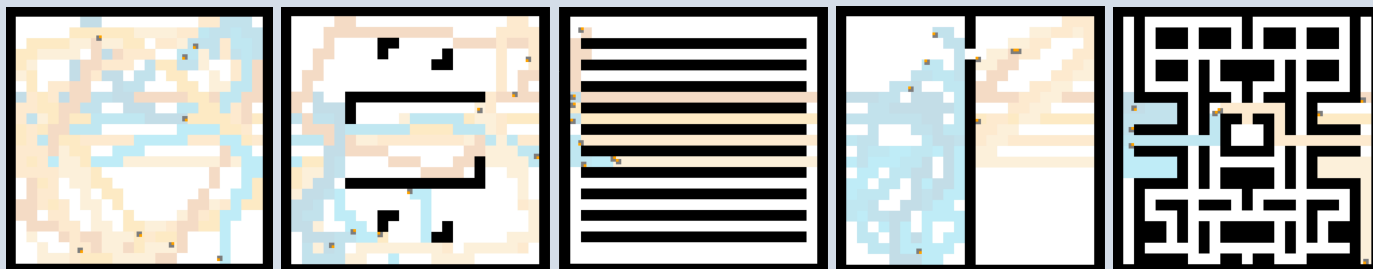
Objectif: capturer un maximum de cases !
Une case appartient à l'équipe du dernier robot l'ayant visitée

Démo



```
python tetracomposibot.py config_Paintwars
```

nicolas.bredecche@upmc.fr



nicolas.bredecche@upmc.fr

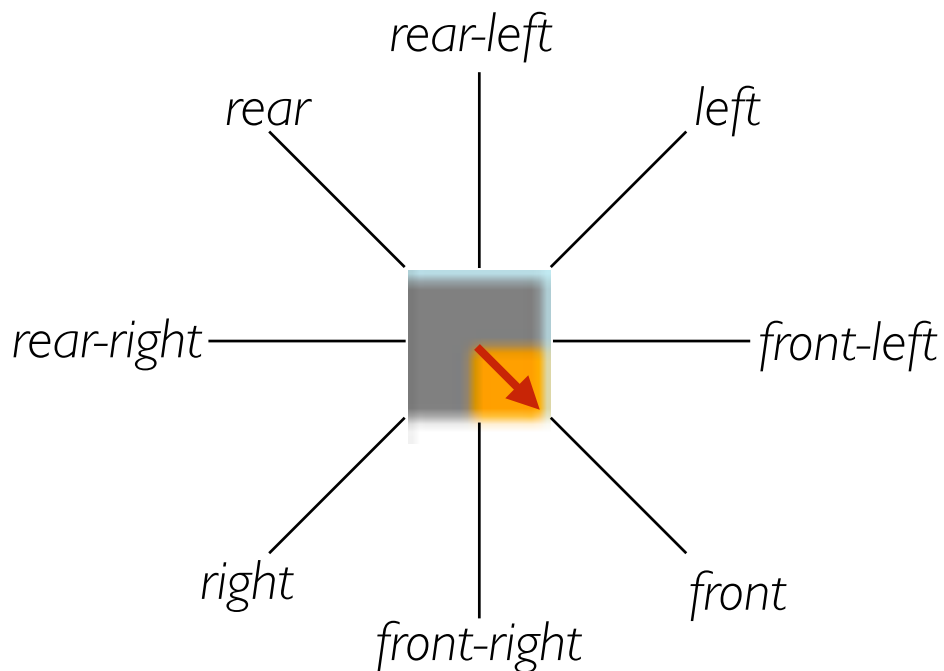


Un robot : empreinte 2x2, orientation en orange

La position et l'orientation sont des valeurs **continues**

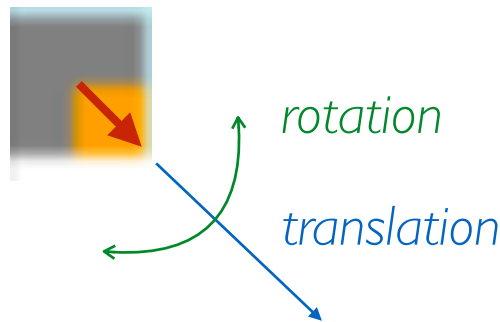
L'affichage et la gestion des collisions sont gérés dans une grille (résolution discrète)

nicolas.bredeche@upmc.fr



Senseurs de proximité

nicolas.bredeche@upmc.fr



Contrôle : *translation* et *rotation*

commande ***translation*** entre -1.0 (recule) et +1.0 (avance)
translation effective bornée par une valeur max (déplacement en pixels)

commande ***rotation*** entre -1.0 (tourne vers la droite) et +1.0 (tourne vers la gauche)
rotation effective bornée par une valeur max (rotation en degré)

nicolas.bredeche@upmc.fr

72

```
from robot import *

class Robot_player(Robot):

    def __init__(self, x_0, y_0, theta_0, name="n/a", team="n/a"):
        super().__init__(x_0, y_0, theta_0, name=name, team=team)

    def step(self, sensors, sensor_view=None, sensor_robot=None, sensor_team=None):
        translation = sensors[sensor_front]*0.8
        rotation = 1.0 * sensors[sensor_front_left] - 1.0 * sensors[sensor_front_right] + (random.random()-0.5)*0.1
        return translation, rotation
```

Fonction *step(...)*

Cette fonction est appelée à chaque pas de temps

Translation et rotation: ce sont des **commandes instantanées** (et pas un asservissement)

nicolas.bredeche@upmc.fr

robot_challenger
robot_champion

Challenger

Des robots rapides mais
pas très fins
commence à gauche
traces **bleues**

Champion

Des robots prudents
mais plus précis
commence à droite
traces **rouges**

Votre mission: modifier l'équipe "challenger"

nicolas.bredeche@upmc.fr

```
# Projet "robotique" IA&Jeux 2025
#
# Binome:
# Prénom Nom: _____
# Prénom Nom: _____
#
# check robot.py for sensor naming convention
# all sensor and motor value are normalized (from 0.0 to 1.0 for sensors, -1.0 to +1.0 for motors)

from robot import *

nb_robots = 0

class Robot_player(Robot):
    team_name = "Challenger"
    robot_id = -1
    memory = 0 # vous n'avez le droit qu'a une case mémoire qui doit être obligatoirement un entier

    def __init__(self, x_0, y_0, theta_0, name="n/a", team="n/a"):
        global nb_robots
        self.robot_id = nb_robots
        nb_robots+=1
        super().__init__(x_0, y_0, theta_0, name="Robot "+str(self.robot_id), team=self.team_name)

    def step(self, sensors, sensor_view=None, sensor_robot=None, sensor_team=None):
        translation = sensors[sensor_front]
        rotation = 1.0 * sensors[sensor_front_left] - 1.0 * sensors[sensor_front_right] + (random.random()-0.5)*0.1
        return translation, rotation
```

robot_challenger.py — en rouge, les parties modifiables. Le reste ne doit pas être modifié.

nicolas.bredeche@upmc.fr

```
team_name = "Challenger"
robot_id = -1
memory = 0
```

(...)

doit être modifié — choisissez votre nom!
ne pas modifier
une seule case mémoire autorisée (un entier)

```
def step(self, sensors, sensor_view=None, sensor_robot=None, sensor_team=None):
    (...)
```

```
print ("Robot",self.robot_id,"(team "+str(self.team_name)+")","at step",self.iteration,":")
print ("\tsensors (distance, max is 1.0) =",sensors)
print ("\ttype (0:empty, 1:wall, 2:robot) =",sensor_view)
print ("\trobot's name (if relevant)      =",sensor_robot)
print ("\trobot's team (if relevant)       =",sensor_team)
```

(...)

sensors: renvoie la **distance** à l'obstacle (pas d'obstacle: 1.0)

sensor_view: renvoie le **type de l'obstacle** touché (0: rien, 1: mur, 2: robot)

sensor_robot: renvoie le **numéro du robot** touché (si c'est un robot)

sensor_team: renvoie le **nom de l'équipe du robot** touché (si c'est un robot)

Ce sont des vecteurs de taille 8 (une valeur par senseur)

```
sensor_front = 0
sensor_front_left = 1
sensor_left = 2
sensor_rear_left = 3
sensor_rear = 4
sensor_rear_right = 5
sensor_right = 6
sensor_front_right = 7
```

```
python tetracomposibot.py config_Paintwars
```

fichier config
sans l'extension .py
cf. en particulier les 3 premiers paramètres
(valeurs par défaut pour l'arène, la position initiale et le rendu)

rendu
0: normal
1: rapide
2: rapide, sans affichage

numéro de l'arène
cf. arenas.py

```
python tetracomposibot.py config_Paintwars 1 False 0
```

fichier config
sans l'extension .py

position initiale
classique ou inversée

- Le jour de l'évaluation

- ▶ interview avec votre enseignant·e
 - test sur les 5 arènes (en partant à gauche, puis à droite)
 - test sur 5 nouvelles arènes originales (différentes selon les groupes de TP)
 - présentation de vos stratégies (expliquer les points essentiels)
 - par exemple: architecture de Subsumption pour combiner des comportements simples
 - un robot (minimum) doit implémenter une stratégie obtenue par évolution
(attention, il ne s'agit pas de faire une évolution en temps réel)
- ▶ tournoi entre les équipes
 - test sur les 10 arènes
 - deux points de départ: à droite et à gauche
 - tableau des scores — toutes les équipes s'affrontent!
- ▶ dépôt de votre code sur Moodle
 - n'oubliez pas de mettre vos noms dans le code (cf. en haut du fichier)

Conclusions



- Vous êtes prêt pour programmer un robot réactif!
 - ▶ Véhicules de Braitenberg
 - ▶ Machines à états finis
 - ▶ Arbres de décision
 - ▶ Arbres de comportements
 - ▶ Architecture de Subsumption
- Prochain TME
 - ▶ première partie : application de ce cours (noté)
 - ▶ seconde partie : travail sur le projet
- Au prochain cours
 - ▶ comment automatiser la conception de comportements

nicolas.bredeche@upmc.fr

Bibliographie: articles cités

- M.J. Mataric (1994) Interaction and Intelligent Behavior. Ph.D. MIT.
- R.A. Brooks (1986) A robust layered control system for a mobile robot. IEEE journal on robotics and automation
- V. Braitenberg (1986) Vehicles: Experiments in synthetic psychology. MIT press.
- C.W. Reynolds (1987) Flocks, herds and schools: A distributed behavioral model. ACM SIGGRAPH comp. graph.
- T. Gomi, A. Griffith (1998) Developing intelligent wheelchairs for the handicapped. Assistive Tech. and AI.
- R.A.Brooks, A.M. Flynn (1989) Fast, cheap and out of control. MIT AI memo #1182.
- I.D. Couzin et al. (2002) Collective memory and spatial sorting in animal groups. Journal of theoretical biology 218.1: 1-11.
- M. Dorigo, V. Maniezzo, A. Coloni (1996) Ant system: optimization by a colony of cooperating agents. IEEE TSMC part B
- R.R. Murphy (2000) An Introduction to AI Robotics. Bradford Book.
- R.A. Arkin (2000) Behavior-Based Robotics (3rd printing). Bradford Book.
- B. Siciliano, O. Khatib, editors (2016) Handbook of robotics. Springer.
- S.B. Niku (2010) Introduction to Robotics: Analysis, Control, Applications (2nd Edition). Wiley.
- M. Colledanchis, P. Ögren (2016) How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions, the Subsumption Architecture, and Decision Trees. IEEE Transactions on Robotics.